

Homework 03*Due: August 24, 2026 at 11:59 PT**August 19, 2026***Sources**

- Book: <https://dsc40b.com/materials/default/notes/book.pdf>
- Lecture 08: https://dsc-courses.github.io/DSC40B-SU26/resources/lectures/lec08/Lecture_08_blank.pdf
- Lecture 09: https://dsc-courses.github.io/DSC40B-SU26/resources/lectures/lec09/Lecture_09_blank.pdf
- Lecture 10: https://dsc-courses.github.io/DSC40B-SU26/resources/lectures/lec10/Lecture_10_blank.pdf

Instructions

Problem 1 is a single continuous simulation: its parts are not independent, and the warning at the head of that problem describes how the parts feed one another. Every trace must show the probe sequence, not just the resulting table; a table with no supporting sequence earns no credit. Report probe counts using the lecture convention, in which the inspection of the base address counts as probe 1. All arithmetic is exact integer arithmetic; do not round.

Problem 1

One Table, Every Policy.

Read this before starting. Parts (a) through (c) simulate *one* hash table over its lifetime. There is a single array of $m = 11$ slots, and it is created empty exactly once, at the start of part (a). It is **never emptied**. Each part inherits the table exactly as the previous part left it, and every insertion is permanent. Part (d) then rebuilds the keys of that table into a cuckoo structure.

Consequently, insertions on each of the parts are **not independently gradeable**. If a key lands in the wrong slot in part (a), every probe sequence in parts (b) through (d) that passes through that slot (and doesn't get lucky to land in its correct position) is also wrong, and so those points are lost as well. Verify each part before moving to the next. Queries (lookups, probe counts, load factors, and every question that only reads the table) do *not* modify it; only insertion does.

The table, the key set, and the hash family are fixed once, below, and used by every part. The only thing that changes from part to part is a single integer κ derived from that part's own letter, and the collision-resolution policy that part is told to use.

Setup

The table. A single open-addressed hash table with $m = 11$ slots, indexed $0, 1, \dots, 10$. Note that 11 is prime. Every slot is either **EMPTY** or occupied by a key.

The dataset. Eight keys, indexed $D[0]$ through $D[7]$ starting from zero. Each part is told which contiguous block of D it inserts; the order within a block is left to right.

$$D = \langle \underbrace{713, 738}_{\text{inserted in (a)}}, \underbrace{106, 297}_{\text{inserted in (b)}}, \underbrace{626, 275}_{\text{inserted in (c)}}, \underbrace{409, 331}_{\text{inserted in (d)}} \rangle$$

The family. Fix the prime $p = 1009$. The family is $\mathcal{H} = \{h_0, h_1, \dots, h_7\}$, where

$$h_j(k) = ((a_j \cdot k + b_j) \bmod p) \bmod m, \quad j \in \{0, 1, \dots, 7\},$$

with the constants:

j	0	1	2	3	4	5	6	7
a_j	3	11	17	29	41	53	67	79
b_j	7	2	23	5	13	31	19	3

The selector. Which member of $\mathcal{H}$ a key uses is itself decided by a hash function. Let σ be the polynomial string hash from Lecture 08, applied to the *decimal representation* of the key with multiplier $x = 31$:

```

1 def sigma(k):
2     h = 0
3     for ch in str(k):           # decimal digits, left to right
4         h = (31 * h + ord(ch)) % 8
5     return h

```

Recall $\text{ord}('0') = 48$, $\text{ord}('1') = 49$, and so on.

The per-part offset. This problem has four *parts*, labelled by the letters (a) through (d). Each part is subdivided into numbered *items* (1), (2), (3), and so on. The offset is derived from the **letter of the part**, never from the number of an item:

$$\kappa_\ell = (11 \cdot \text{ord}(\ell) + 5) \bmod 23, \quad \ell \in \{\mathbf{a}, \mathbf{b}, \mathbf{c}, \mathbf{d}\}.$$

The base address. A key inserted during part ℓ has base address

$$\sigma_\ell(k) = (\sigma(k) + \kappa_\ell) \bmod 8, \quad H_\ell(k) = h_{\sigma_\ell(k)}(k),$$

where H_ℓ plays the role $h(k)$ played in lecture. Note the division of labour: the *letter* fixes κ_ℓ once for the whole part, and then the *key* selects which of the eight family members it uses, via $\sigma_\ell(k)$. Two keys inserted in the same part will generally use different members of $\mathcal{H}$. **A key keeps the offset it was inserted under for the rest of its life.** The table therefore records, next to each key, the letter of the part that inserted it, and any later lookup of that key reproduces its original probe sequence using *that* letter's κ and *that* letter's probing rule.

Conventions. $\alpha = n/m$ is the load factor, computed from the number of keys in the table. A blank $m = 11$ diagram is supplied wherever a table state is requested; reproduce it in your submission.

(a) The first two keys**[20 points]**

The table is empty. Using κ_a , insert $D[0]$ and $D[1]$ under **linear probing**,

$$H(k, i) = (H_a(k) + i) \bmod m, \quad i = 0, 1, 2, \dots$$

- (1) **[5 points]** Compute $\sigma(k)$ for all eight keys of D . Show the running value of h after each character for at least two of the keys.
- (2) **[10 points]** Perform the two insertions. Give a trace table with one row per key and columns: k , $\sigma_a(k)$, $H_a(k)$, the full sequence of slots inspected, the landing slot, and the number of probes. Then fill in the table diagram. Your trace table should look something like this, which will be reused in every part:

k	$\sigma_a(k)$	$H_a(k)$	slots inspected	landing slot	probes
713					
738					

State of the table at the end of part (a)

slot	0	1	2	3	4	5	6	7	8	9	10
key											

- (3) **[5 points]** Report α after these insertions, the total number of probes spent, and the average probes per insertion.

(b) Quadratic probing into the occupied table**[30 points]**

Carry the table forward from part (a). Using κ_b , set $c_1 = \kappa_b \bmod 4$ and $c_2 = 1$, and insert $D[2]$ and $D[3]$ under **quadratic probing**,

$$H(k, i) = (H_b(k) + c_1 i + c_2 i^2) \bmod m, \quad i = 0, 1, 2, \dots$$

- (1) **[5 points]** State κ_b , c_1 , and c_2 explicitly.
- (2) **[10 points]** Perform the two insertions. Give a trace table with columns k , $\sigma_b(k)$, $H_b(k)$, the offset $c_1 i + c_2 i^2$ at each step, the slot inspected, the landing slot, and the number of probes. Then fill in the table diagram.
- (3) **[15 points]** Determine whether any key in this batch was turned away from a slot occupied by a key that part (a) inserted. If so, name each such pair, give the slot where they met, and state which of the two occupies it; if not, say so and state which slots this batch inspected that were free. Either way, say in one sentence what would have been different for this batch had the table been emptied first.

State of the table at the end of part (b)

slot	0	1	2	3	4	5	6	7	8	9	10
key											

(c) Double hashing into the occupied table**[25 points]**

Carry the table forward from part (b). Using κ_c , insert $D[4]$ and $D[5]$ under **double hashing**. The step is drawn from a different member of the same family, offset by four:

$$H(k, i) = (H_c(k) + i \cdot g(k)) \bmod m, \quad g(k) = 1 + \left(h_{(\sigma_c(k)+4) \bmod 8}(k) \bmod 10 \right).$$

- (1) **[10 points]** Perform the two insertions. Give a trace table with columns k , $\sigma_c(k)$, $H_c(k)$, $g(k)$, the full probe sequence, the landing slot, and the number of probes. Then fill in the table diagram.
- (2) **[5 points]** Determine whether any key in this batch was turned away from more than an occupied slot before it landed. List every such key, and for each slot it was turned away from give the occupying key and the letter of the part that inserted that key.
- (3) **[5 points]** List every pair of keys currently in the table that share a base address under their own insertion offsets. For each pair, give the step size each key's own probing rule uses and the probe index at which their two sequences first diverge, or state that they never diverge. Then explain in one sentence what double hashing buys for such a pair that linear and quadratic probing do not.
- (4) **[5 points]** Suppose g were defined as $g(k) = h_{(\sigma_c(k)+4) \bmod 8}(k)$, dropping the " $1 + (\cdot \bmod 10)$ " wrapper. Identify the specific failure this introduces and name a key in D that triggers it.

State of the table at the end of part (c)

slot	0	1	2	3	4	5	6	7	8	9	10
key											

(d) Cuckoo hashing**[25 points]**

The table from part (c) is now rebuilt as a cuckoo structure. Build *two* fresh tables T_1 and T_2 , each with $m = 11$ slots. Take the keys of the part (c) table **in increasing order of the slot they currently occupy** and insert them in that order; then insert $D[6]$ and $D[7]$, in that order. Using κ_d , define

$$f(k) = h_{\sigma_d(k)}(k) \quad (\text{position in } T_1), \quad g(k) = h_{(\sigma_d(k)+4) \bmod 8}(k) \quad (\text{position in } T_2).$$

Insertion follows the lecture rule: place k at $T_1[f(k)]$; if occupied, evict the current occupant k' , put k there, and place k' at $T_2[g(k')]$; if that is occupied, evict its occupant and send it back to T_1 , alternating. If a single insertion reaches m evictions, declare a rehash.

- (1) **[5 points]** State the insertion order explicitly, as a list of eight keys: the six carried over from part (c), each labelled with the part (c) slot it came from, followed by $D[6]$ and $D[7]$. An error here invalidates the entire part.
- (2) **[6 points]** Give a trace table with columns k , $\sigma_d(k)$, $f(k)$, and $g(k)$ for every key in that list.
- (3) **[14 points]** Perform the insertions. For each, give the displacement chain in the form $k \rightarrow (T_1, s) \rightarrow k' \rightarrow (T_2, s') \rightarrow \dots$ and the number of evictions. Then fill in both table diagrams.

Part (d): T_1

slot	0	1	2	3	4	5	6	7	8	9	10
key											

Part (d): T_2

slot	0	1	2	3	4	5	6	7	8	9	10
key											